Learning Objectives

Learners will be able to...

- Define Natural Language Processing (NLP) and understand its usage
- Install NLTK using pip
- Access various corpora and lexicons, as well as utilize reader and wordnet functions
- Tokenize words and sentences
- Access text from the web, from local files, and user input

---

info

Make Sure To Know

Intermediate Python

Limitations

In this assignment, we only work with the NLTK library.

Use NLTK for NLP

Definition: Natural Language Processing

Natural Language Processing (NLP) lies at the intersection of linguistics, computer science, and artificial intelligence. Its focus is to give computers the power to read a written text and interpret spoken words just as humans can. This is done through the use of computational modeling of human language, which allows real-time analysis of data.

Some common forms of NLP that you may use on a regular basis:

- **Chatbots** - chatbots are used for customer service or even to help you learn a new language.
- **Virtual assistants** - assistants from Amazon, Apple, and Google allow you to interact with computing devices in a natural manner.
- **Online translation** - computers can properly translate text by understanding the larger context through NLP.
- **Spam checkers** - NLP can identify words and phrases that frequently suggest spam or a phishing attempt.

Install NLTK

While there are many different ways to implement NLP, this course uses the Natural Language Toolkit (NLTK) library. Using the NLTK package, we can pre-process text before analyzing it further. Note that NLTK can interpret English, but does not fully support the interpretation of other natural languages, such as Spanish.

Enter the below code in the Terminal located on the left-hand side of the screen to install NLTK.

```
python3 -m pip install nltk==3.6.7
```

Before moving on, verify that `nltk` is properly installed.

challenge

Verify the installation:

Entering the below code lists all of the Python modules currently installed on your system.

```
python3 -m pip list
```

You should see a version of `nltk` in the list of modules, as shown below.

```
nltk (3.6.7)
```

Download NLTK Data

After installing the NLTK library onto our system, we are able to call the `nltk` module and download its data (which includes models, grammars, and other information) onto whichever Python file we will be using for NLP.

For this exercise, we will not work with a `.py` file. Rather, we will use the Python interpreter in the Terminal by entering the below code. This starts the Python interactive shell, which acts as if we are working with a `.py` file.

```
python3
```

After entering this command, the Terminal will display a `>>>` as a prompt. We will enter code at each of these prompts and press `ENTER` to run each line.

Use the below code to import the `nltk` module and then call the `download` method.

```
import nltk
nltk.download()
```

The `download` method calls the download program, displaying the following menu:

```
NLTK Downloader
---------------------------------------------------------------------------
    d) Download   l) List    u) Update   c) Config   h) Help   q) Quit
---------------------------------------------------------------------------
Downloader>
```

- Enter `d` for download.
- Then enter `all` to download the entirety of the NLTK library's data.
- After downloading has finished, enter `q` to quit the download program.

▼
nltk.download() explained

```
import nltk
nltk.download()
```

The above command we just ran downloads all data from the NLTK library. If we need to keep the size of our program to a minimal, we also have the ability to download subsets of the available data. See NLTK Documentation for more information.

challenge

Verify the download:

To test if the `nltk.download()` command was successfully executed in our Terminal's Python interpreter, we can import the *brown corpus*, a pre-defined file, and then return a list of strings from that file using the *reader function* called `words`.

```
from nltk.corpus import brown
brown.words()
```

Your output should display as shown below.

```
['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', ...]
```

Access Corpora

Definitions: Corpus & Corpora

- The word *corpus* refers to one collection of texts.
- The word *corpora* refers to multiple collections of texts.
- We use the `nltk.corpus` package from the NLTK library to import corpora and apply corpus reader functions.

The corpora included in the NLTK library contain several genres of text.

Access Corpora

There are several corpora contained in the NLTK library. The most commonly used are listed and described below.
- **The Brown Corpus**: Created by Brown University in 1961, this was the first million-word electronic corpus of English.
- **The Gutenberg Corpus**: Containing 25,000 free electronic books, this data was taken from the Project Gutenberg electronic text archive of literature.
- **The Web Text Corpus**: Containing less formal language, this data was taken from an online discussion forum, personal advertisements and reviews, and so on.
- **The NPS Chat Corpus**: Originally created by the Naval Postgraduate School, this corpus contains over 10,000 posts from instant messaging chats.
- **The Reuters Corpus**: This corpus contains over 10,000 news documents, grouped into two sets called 'training' and 'test'.
- **The Inaugural Address Corpus**: This corpus contains each presidential inaugural address.

Enter the code below to access each of the above-mentioned corpora.

```
from nltk.corpus import brown
from nltk.corpus import gutenberg
from nltk.corpus import webtext
from nltk.corpus import nps_chat
from nltk.corpus import reuters
from nltk.corpus import inaugural
```

Use Corpus Reader Functions

We can use corpus reader functions to display corpus files in different arrangements. The most commonly used are listed and described below.

Enter the code for each reader function to see its functionality displayed, then select the `TRY IT` button to run the file.

.words(): Returns the contents of a specified corpus in a list of strings, where each string element in the list represents a word.

```
print(f'\nUsing words() on the Brown Corpus: {brown.words()}')
```

Your output should display as shown below.

```
Using words() on the Brown Corpus: ['The', 'Fulton', 'County',
'Grand', 'Jury', 'said', ...]
```

.sents(): Returns the contents of a specified corpus in a list of sub lists of strings, where each sub list is grouped by sentence and each string element in each sub list represents a word in the given sentence.

```
print(f'\nUsing sents(): {brown.sents()}')
```

Your output should display as shown below.

```
Using sents(): [['The', 'Fulton', 'County', 'Grand', 'Jury',
'said', 'Friday', 'an', 'investigation', 'of', "Atlanta's",
'recent', 'primary', 'election', 'produced', '``', 'no',
'evidence', "''", 'that', 'any', 'irregularities', 'took',
'place', '.'], ['The', 'jury', 'further', 'said', 'in', 'term-
end', 'presentments', 'that', 'the', 'City', 'Executive',
'Committee', ',', 'which', 'had', 'over-all', 'charge', 'of',
'the', 'election', ',', '``', 'deserves', 'the', 'praise',
'and', 'thanks', 'of', 'the', 'City', 'of', 'Atlanta', "''",
'for', 'the', 'manner', 'in', 'which', 'the', 'election', 'was',
'conducted', '.'], ...]
```

.paras(): Returns the contents of a specified corpus in a list of sub lists of sub lists of strings, where each sub list is grouped by paragraph, each sub list within that sub list is grouped by sentence, and each string element in each sub list of sub lists represents a word in the given sentence in the given paragraph.

```
print(f'\nUsing paras(): {brown.paras()}')
```

Your output should display as shown below.

```
Using paras(): [[['The', 'Fulton', 'County', 'Grand', 'Jury',
'said', 'Friday', 'an', 'investigation', 'of', "Atlanta's",
'recent', 'primary', 'election', 'produced', '``', 'no',
'evidence', "''", 'that', 'any', 'irregularities', 'took',
'place', '.']], [['The', 'jury', 'further', 'said', 'in', 'term-
end', 'presentments', 'that', 'the', 'City', 'Executive',
'Committee', ',', 'which', 'had', 'over-all', 'charge', 'of',
'the', 'election', ',', '``', 'deserves', 'the', 'praise',
'and', 'thanks', 'of', 'the', 'City', 'of', 'Atlanta', "''",
'for', 'the', 'manner', 'in', 'which', 'the', 'election', 'was',
'conducted', '.']], ...]
```

.tagged_words(): Returns the contents of a specified corpus in a list of sub tuples of strings, where each sub tuple contains two string elements - the first string element being the word from the text and the second element being the given word's type of speech tag.

```
print(f'\nUsing tagged_words(): {brown.tagged_words()}')
```

Your output should display as shown below.

```
Using tagged_words(): [('The', 'AT'), ('Fulton', 'NP-TL'), ...]
```

.tagged.sents(): Returns the contents of a specified corpus in a list of sub lists of sub tuples of strings, where each sub list is grouped by sentence, and each sub tuple contains two string elements - the first string element being the word from the text and the second element being the given word's type of speech tag.

```
print(f'\nUsing tagged_sents(): {brown.tagged_sents()}')
```

Your output should display as shown below.

```
Using tagged_sents(): [[('The', 'AT'), ('Fulton', 'NP-TL'),
('County', 'NN-TL'), ('Grand', 'JJ-TL'), ('Jury', 'NN-TL'),
('said', 'VBD'), ('Friday', 'NR'), ('an', 'AT'),
('investigation', 'NN'), ('of', 'IN'), ("Atlanta's", 'NP$'),
('recent', 'JJ'), ('primary', 'NN'), ('election', 'NN'),
('produced', 'VBD'), ('``', '``'), ('no', 'AT'), ('evidence',
'NN'), ("''", "''"), ('that', 'CS'), ('any', 'DTI'),
('irregularities', 'NNS'), ('took', 'VBD'), ('place', 'NN'),
('.', '.')], [('The', 'AT'), ('jury', 'NN'), ('further', 'RBR'),
('said', 'VBD'), ('in', 'IN'), ('term-end', 'NN'),
('presentments', 'NNS'), ('that', 'CS'), ('the', 'AT'), ('City',
'NN-TL'), ('Executive', 'JJ-TL'), ('Committee', 'NN-TL'), (',',
','), ('which', 'WDT'), ('had', 'HVD'), ('over-all', 'JJ'),
('charge', 'NN'), ('of', 'IN'), ('the', 'AT'), ('election',
'NN'), (',', ','), ('``', '``'), ('deserves', 'VBZ'), ('the',
'AT'), ('praise', 'NN'), ('and', 'CC'), ('thanks', 'NNS'),
('of', 'IN'), ('the', 'AT'), ('City', 'NN-TL'), ('of', 'IN-TL'),
('Atlanta', 'NP-TL'), ("''", "''"), ('for', 'IN'), ('the',
'AT'), ('manner', 'NN'), ('in', 'IN'), ('which', 'WDT'), ('the',
'AT'), ('election', 'NN'), ('was', 'BEDZ'), ('conducted',
'VBN'), ('.', '.')], ...]
```

.tagged_paras(): Returns the contents of a specified corpus in a list of sub lists of sub lists of sub tuples of strings, where each sub list is grouped by paragraph, each sub list within a given sub list is grouped by sentence, and each sub tuple contains two string elements - the first string element being the word from the text and the second element being the given word's type of speech tag.

```
print(f'\nUsing tagged_paras(): {brown.tagged_paras()}')
```

Your output should display as shown below.

```
Using tagged_paras(): [[[('The', 'AT'), ('Fulton', 'NP-TL'),
('County', 'NN-TL'), ('Grand', 'JJ-TL'), ('Jury', 'NN-TL'),
('said', 'VBD'), ('Friday', 'NR'), ('an', 'AT'),
('investigation', 'NN'), ('of', 'IN'), ("Atlanta's", 'NP$'),
('recent', 'JJ'), ('primary', 'NN'), ('election', 'NN'),
('produced', 'VBD'), ('``', '``'), ('no', 'AT'), ('evidence',
'NN'), ("''", "''"), ('that', 'CS'), ('any', 'DTI'),
('irregularities', 'NNS'), ('took', 'VBD'), ('place', 'NN'),
('.', '.')]], [[('The', 'AT'), ('jury', 'NN'), ('further',
'RBR'), ('said', 'VBD'), ('in', 'IN'), ('term-end', 'NN'),
('presentments', 'NNS'), ('that', 'CS'), ('the', 'AT'), ('City',
'NN-TL'), ('Executive', 'JJ-TL'), ('Committee', 'NN-TL'), (',',
','), ('which', 'WDT'), ('had', 'HVD'), ('over-all', 'JJ'),
('charge', 'NN'), ('of', 'IN'), ('the', 'AT'), ('election',
'NN'), (',', ','), ('``', '``'), ('deserves', 'VBZ'), ('the',
'AT'), ('praise', 'NN'), ('and', 'CC'), ('thanks', 'NNS'),
('of', 'IN'), ('the', 'AT'), ('City', 'NN-TL'), ('of', 'IN-TL'),
('Atlanta', 'NP-TL'), ("''", "''"), ('for', 'IN'), ('the',
'AT'), ('manner', 'NN'), ('in', 'IN'), ('which', 'WDT'), ('the',
'AT'), ('election', 'NN'), ('was', 'BEDZ'), ('conducted',
'VBN'), ('.', '.')]], ...]
```

challenge

Try this variation:

Write the code to produce the following expected output, then select the `TRY IT` button to run the file and check your work.

The expected output is shown below.

```
The Inaugural Corpus broken up by word: ['Fellow', '-',
'Citizens', 'of', 'the', 'Senate', ...]
```

▼ Solution

One possible solution is shown below, where we can return the Inaugural Corpus broken up by word, using the reader function `words`.

```
print(f'\nThe Inaugural Corpus broken up by word:
        {inaugural.words()}')
```

Access Lexicons

Definition: Lexicon

- A lexicon, often referred to as a lexical resource, is a collection of words and/or phrases, such as a set of vocabulary or a dictionary, marked with allied information, such as each given word's part of speech or definition.
- A lexicon is often considered a type of corpus because it represents text data.

There are several lexicons included in the NLTK library.

Access Lexicons

There are several lexicons contained in the NLTK library. Some commonly used lexicons are listed and described below.
- **The Stopwords Corpus**: Words like "me", "has", "also", and "to" are all examples of stop words. Stop words add little meaning to text data. This lexicon corpus contains stop words in English.
- **The Names Corpus**: Categorized by gender, this lexicon corpus contains over 8,000 first names. Female names are stored in the file called `female.txt` and male names are stored in the file called `male.txt`.
- **The CMU Pronouncing Dictionary Corpus**: Based on US English, this lexicon corpus contains the phonetic pronunciations of words. Each phonetic pronunciation is represented with a symbol based on the Arpabet.

Enter the code below to access each of the above-mentioned lexicons.

```
from nltk.corpus import stopwords
from nltk.corpus import names
from nltk.corpus import cmudict
```

Use `stopwords`

For this first exercise, we will work with the lexicon corpus called `stopwords`. Using the corpus reader function `words`, we can return a list of strings representing English stop words.

Enter the code below, then select the `TRY IT` button to run the file.

```
print(f"\nStop Words in English: {stopwords.words('english')}")
```

Your output should display as shown below.

```
Stop Words in English: ['i', 'me', 'my', 'myself', 'we', 'our',
'ours', 'ourselves', 'you', "you're", "you've", "you'll",
"you'd", 'your', 'yours', 'yourself', 'yourselves', 'he', 'him',
'his', 'himself', 'she', "she's", 'her', 'hers', 'herself',
'it', "it's", 'its', 'itself', 'they', 'them', 'their',
'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this',
'that', "that'll", 'these', 'those', 'am', 'is', 'are', 'was',
'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having',
'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and', 'but',
'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by',
'for', 'with', 'about', 'against', 'between', 'into', 'through',
'during', 'before', 'after', 'above', 'below', 'to', 'from',
'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under',
'again', 'further', 'then', 'once', 'here', 'there', 'when',
'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few',
'more', 'most', 'other', 'some', 'such', 'no', 'nor', 'not',
'only', 'own', 'same', 'so', 'than', 'too', 'very', 's', 't',
'can', 'will', 'just', 'don', "don't", 'should', "should've",
'now', 'd', 'll', 'm', 'o', 're', 've', 'y', 'ain', 'aren',
"aren't", 'couldn', "couldn't", 'didn', "didn't", 'doesn',
"doesn't", 'hadn', "hadn't", 'hasn', "hasn't", 'haven',
"haven't", 'isn', "isn't", 'ma', 'mightn', "mightn't", 'mustn',
"mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn',
"shouldn't", 'wasn', "wasn't", 'weren', "weren't", 'won',
"won't", 'wouldn', "wouldn't"]
```

challenge

Try this variation:

With the use of the `stopwords` lexicon corpus, we can find the percentage of how often English stop words were used in Inaugural Addresses stored in the `inaugural` corpus.

Enter the code below, then select the `TRY IT` button to run the file.

```
import nltk
inaugural_corpus_words = nltk.corpus.inaugural.words()

def stopwords_percentage(data):
  english_stopwords = stopwords.words('english')
  content = [element for element in data if element.lower()
        in english_stopwords]
  return(100 * (len(content) / len(data)))

print(f'\nPercentage of Stop Words in the Inaugural Corpus:
        {stopwords_percentage(inaugural_corpus_words)}%')
```

Your output should display as shown below.

```
Percentage of Stop Words in the Inaugural Corpus:
47.59746502638962%
```

Use `names`

For this second exercise, we will work with the lexicon corpus called `names`. Using the corpus reader function `words`, we can create a variable called `male_names` to represent all male names contained in this corpus. Then, we can analyze this data, such as by returning all male names that start with a "Za".

Enter the code below, then select the `TRY IT` button to run the file.

```
male_names = names.words('male.txt')

male_names_startingwith_Za = [element for element in male_names
        if element.startswith('Za')]
print(f"\nMale Names Starting with 'Za' in the Names Lexicon
        Corpus : {male_names_startingwith_Za}")
```

Your output should display as shown below.

```
Male Names Starting with 'Za' in the Names Lexicon Corpus :
['Zach', 'Zacharia', 'Zachariah', 'Zacharias', 'Zacharie',
'Zachary', 'Zacherie', 'Zachery', 'Zack', 'Zackariah', 'Zak',
'Zalman', 'Zane', 'Zared', 'Zary']
```

challenge

Try this variation:

Write the code to produce the following expected output, then select the `TRY IT` button to run the file and check your work.

The expected output is shown below.

```
Percentage of Female Names Ending with 'a' in the Names
Lexicon Corpus: 35.45290941811638%
```

▼ **Solution**

One possible solution is shown below. Store all of the female names in a variable. Use the `endswith()` method to find all of the female names that end with `a`. Calculate the percentage of female names ending with `a` as compared to the entire list of female names. Print the result.

```
female_names = names.words('female.txt')

female_names_endingwith_a = [element for element in
        female_names if element.endswith('a')]
female_percentage = 100 * len(female_names_endingwith_a) /
        len(female_names)
print(f"\nPercentage of Female Names Ending with 'a' in the
        Names Lexicon Corpus: {female_percentage}%")
```

Use Wordnet

Definition: Wordnet

- *Wordnet* is an English dictionary used to define single words and phrases.
- In the NLTK library, *Wordnet* represents a lexical corpus.
  - This corpus possesses 155,287 words, along with short definitions and usage examples of each, and 117,659 corresponding groups of words called synonym sets (`synsets`) that relate to each word.

Synsets & Lemmas

Using the `nltk.corpus` package, we can import `wordnet` from the NLTK library.

```
from nltk.corpus import wordnet
```

The imported `wordnet` lexicon corpus will allow us to find the synonym set(s) of a specified word. Enter the code below, then select the `TRY IT` button to run the file.

```
hello_synset = wordnet.synsets('hello')
print(f"\nThe Synonym Set for the word 'hello' is:
        {hello_synset}")
```

Your output should display as shown below:

```
The Synonym Set for the word 'hello' is: [Synset('hello.n.01')]
```

▼ **Did you notice?**

```
The Synonym Set for the word 'hello' is: [Synset('hello.n.01')]
```

There is only one synonym set for the word `hello`. Note that this is not always the case, as words often have several corresponding synonym sets.

We can also retrieve the lemmas (lemmas are the names of groups of words, which may have varying forms (e.g.: *walk* and *walked*), that all refer to the same particular meaning) of synonym sets.

Enter the code below, then select the `TRY IT` button to run the file.

```
hello_lemma_names = wordnet.synset('hello.n.01').lemma_names()
print(f"\nThe Lemma Names in the Synonym Set 'hello.n.01' are:
        {hello_lemma_names}")
```

Your output should display as shown below:

```
The Lemma Names in the Synonym Set 'hello.n.01' are: ['hello',
'hullo', 'hi', 'howdy', 'how-do-you-do']
```

We can find the lemmas of a specified word, such as `'hello'`, and we can also find the lemmas of a specified synonym set, such as `'hello.n.01'`. Enter the code below, then select the `TRY IT` button to run the file.

```
hello_lemmas =  wordnet.lemmas('hello')
print(f"\nThe Lemmas in the Word 'hello' are: {hello_lemmas}")

hello_n_01_synset_lemmas = wordnet.synset('hello.n.01').lemmas()
print(f"\nThe Lemmas in the Synonym Set 'hello.n.01' are:
        {hello_n_01_synset_lemmas}")
```

Your output should display as shown below:

```
The Lemmas in the Word 'hello' are: [Lemma('hello.n.01.hello')]
The Lemmas in the Synonym Set 'hello.n.01' are:
[Lemma('hello.n.01.hello'), Lemma('hello.n.01.hullo'),
Lemma('hello.n.01.hi'), Lemma('hello.n.01.howdy'),
Lemma('hello.n.01.how-do-you-do')]
```

We can even go in the reverse direction, and retrieve the synset and synset name of a specified lemma, such as `'hello.n.01.hello'`.

Enter the code below, then select the `TRY IT` button to run the file.

```
hello_n_01_hello_lemma_synset =
        wordnet.lemma('hello.n.01.hello').synset()
print(f"\nThe Synset for the Lemma 'hello.n.01.hello' is:
        {hello_n_01_hello_lemma_synset}")


hello_n_01_hello_lemma_synset_name =
        wordnet.lemma('hello.n.01.hello').name()
print(f"\nThe Word / Synset Name for the Lemma
        'hello.n.01.hello' is:
        {hello_n_01_hello_lemma_synset_name}")
```

Your output should display as shown below:

```
The Synset for the Lemma 'hello.n.01.hello' is:
Synset('hello.n.01')
The Word / Synset Name for the Lemma 'hello.n.01.hello' is:
hello
```

challenge

Try this variation:

As we said before, not every word has only one corresponding synonym set. Often, words have many synonym sets, as we see with the word `'canine'`. The function defined below will print all lemma names in each synonym set for the word `'canine'`.

Enter the below code, then select the `TRY IT` button to run the file.

```
canine_synset = wordnet.synsets('canine')
print(f"\nThe Synonym Sets for the word 'canine' are:
        {canine_synset}")


for element in canine_synset:
  element_synset = str(element)[8:-2]
  print(f"\nThe Lemmas in the Synonym Set '{element_synset}'
        are: {element.lemma_names()}")
```

Your output should display as shown below.

```
The Synonym Sets for the word 'canine' are:
[Synset('canine.n.01'), Synset('canine.n.02'),
Synset('canine.a.01'), Synset('canine.a.02')]
The Lemmas in the Synonym Set 'canine.n.01' are: ['canine',
'canine_tooth', 'eyetooth', 'eye_tooth', 'dogtooth',
'cuspid']
The Lemmas in the Synonym Set 'canine.n.02' are: ['canine',
'canid']
The Lemmas in the Synonym Set 'canine.a.01' are: ['canine',
'laniary']
The Lemmas in the Synonym Set 'canine.a.02' are: ['canine']
```

More Uses of Wordnet

We can also retrieve the definitions of specified synonym sets, as well as their example usages.

Enter the code below, then select the `TRY IT` button to run the file.

```
hello_def = wordnet.synset('hello.n.01').definition()
print(f"\nThe Definition for the Synonym Set 'hello.n.01' from the word
        'hello' is: {hello_def}")


hello_examples = wordnet.synset('hello.n.01').examples()
print(f"\nExamples for the Synonym Set 'hello.n.01' from the word 'hello'
        is: {hello_examples}")
```

Your output should display as shown below:

```
The Definition for the Synonym Set 'hello.n.01' from the word 'hello' is: an
expression of greeting

Examples for the Synonym Set 'hello.n.01' from the word 'hello' is: ['every
morning they exchanged polite hellos']
```

challenge

Try this variation:

Hyponyms are also included in the `wordnet` lexicon corpus. Hyponyms represent types of a specified item.

- For example, the word *dog* is a hyponym of the word *animal*.

Enter the below code, then select the `TRY IT` button to run the file.

```
vehicle = wordnet.synset('vehicle.n.01')
types_of_vehicles = vehicle.hyponyms()
print(f'\nHyponyms / Types of Vehicles: {types_of_vehicles}')

wheeled_vehicle_n_01_hyponyms = wordnet.synset('wheeled_vehicle.n.01').hyponyms()
print(f"\nHyponyms of Hyponym 'wheeled_vehicle_n_01': {wheeled_vehicle_n_01_hyponyms}")
```

Your output should display as shown below.

```
Hyponyms / Types of Vehicles:  [Synset('bumper_car.n.01'), Synset('craft.n.02'),
Synset('military_vehicle.n.01'), Synset('rocket.n.01'), Synset('skibob.n.01'),
Synset('sled.n.01'), Synset('steamroller.n.02'), Synset('wheeled_vehicle.n.01')]

Hyponyms of Hyponym 'wheeled_vehicle_n_01': [Synset('baby_buggy.n.01'),
Synset('bicycle.n.01'), Synset('boneshaker.n.01'), Synset('car.n.02'),
Synset('handcart.n.01'), Synset('horse-drawn_vehicle.n.01'),
Synset('motor_scooter.n.01'), Synset('rolling_stock.n.01'), Synset('scooter.n.02'),
Synset('self-propelled_vehicle.n.01'), Synset('skateboard.n.01'),
Synset('trailer.n.04'), Synset('tricycle.n.01'), Synset('unicycle.n.01'),
Synset('wagon.n.01'), Synset('wagon.n.04'), Synset('welcome_wagon.n.01')]
```

Tokenize Text

Definition: Token

- The process of tokenization involves dividing plain text, such as a phrase, sentence, paragraph, or entire documents into smaller chunks of data so that it is easier to analyze.
- These smaller chunks of data are referred to as *tokens*, which may include words, phrases, or sentences.

There are two methods commonly used to tokenize text: `word_tokenize` and `sent_tokenize`.

Tokenizing by word - The benefit of using tokenization by word is that we can pinpoint words that are frequently used. If we were analyzing a group of restaurant ads in NY, and found that the word “vegan” was used often, then we might assume that there are plenty of vegan options at these restaurants.

```
word_tokenize
```

Tokenizing by sentence - When tokenizing by sentence, we have the ability to analyze how the words in the sentence correlate with each other, which allows us to better understand the context of the words. If we were analyzing a group of restaurant ads in NY, and found that the sentence “No vegan options.” was used, then we can determine that there are not plenty of vegan options at these restaurants.

```
sent_tokenize
```

Word Tokens

Using the `nltk.tokenize` package, we can import the `word_tokenize` method from the NLTK library.

For this first exercise, we will work with a string of data assigned to the variable called `lorenzo_paragraph`, as shown in the code below.

```
from nltk.tokenize import word_tokenize

lorenzo_paragraph = "Lorenzo di Piero de'Medici was an Italian
        statesman, banker, de facto ruler of the Florentine
        Republic and the most powerful and enthusiastic patron
        of Renaissance culture in Italy. Also known as Lorenzo
        the Magnificent (Lorenzo il Magnifico) by contemporary
        Florentines, he was a magnate, diplomat, politician and
        patron of scholars, artists, and poets. As a patron,
        he's best known for his sponsorship of artists such as
        Botticelli and Michelangelo. He held the balance of
        power within the Italic League, an alliance of states
        that stabilized political conditions on the Italian
        peninsula for decades, and his life coincided with the
        mature phase of the Italian Renaissance and the Golden
        Age of Florence."
```

The imported `word_tokenize` method allows us to break this text down by word and punctuation, where each word is grouped in its own string within a list.

Enter the code below, then select the `TRY IT` button to run the file.

```
print(f'Tokenized Text by Word Using word_tokenize():
        {word_tokenize(lorenzo_paragraph)}')
```

Your output should display as shown below:

```
Tokenized Text by Word Using word_tokenize(): ['Lorenzo', 'di',
'Piero', "de'Medici", 'was', 'an', 'Italian', 'statesman', ',',
'banker', ',', 'de', 'facto', 'ruler', 'of', 'the',
'Florentine', 'Republic', 'and', 'the', 'most', 'powerful',
'and', 'enthusiastic', 'patron', 'of', 'Renaissance', 'culture',
'in', 'Italy', '.', 'Also', 'known', 'as', 'Lorenzo', 'the',
'Magnificent', '(', 'Lorenzo', 'il', 'Magnifico', ')', 'by',
'contemporary', 'Florentines', ',', 'he', 'was', 'a', 'magnate',
',', 'diplomat', ',', 'politician', 'and', 'patron', 'of',
'scholars', ',', 'artists', ',', 'and', 'poets', '.', 'As', 'a',
'patron', ',', 'he', "'s", 'best', 'known', 'for', 'his',
'sponsorship', 'of', 'artists', 'such', 'as', 'Botticelli',
'and', 'Michelangelo', '.', 'He', 'held', 'the', 'balance',
'of', 'power', 'within', 'the', 'Italic', 'League', ',', 'an',
'alliance', 'of', 'states', 'that', 'stabilized', 'political',
'conditions', 'on', 'the', 'Italian', 'peninsula', 'for',
'decades', ',', 'and', 'his', 'life', 'coincided', 'with',
'the', 'mature', 'phase', 'of', 'the', 'Italian', 'Renaissance',
'and', 'the', 'Golden', 'Age', 'of', 'Florence', '.']
```

▼ **Did you notice?**

```
[..., 'Piero', "de'Medici", 'was', ...]
```

The computer recognized *de'Medici* as an individual word, rather than two.

```
[..., 'As', 'a', 'patron', ',', 'he', "'s", 'best', 'known',
...]
```

On the other hand, the computer recognized *he's* as two words, rather than one. This is because this library already knows that *he* and *'s* represent two individual words, one being *he* and the other being a contraction of *is*.

Since *de'Medici* is not a known contracted word to the library, it is only recognized as one individual word.

---

challenge

Try this variation:

Let's use the reader function called `words()` previously used in the earlier pages of this assignment to tokenize the variable called `lorenzo_paragraph` by word again.

Predict what you think will happen, enter the below code, then select the `TRY IT` button to run the file.

```
print('\nTokenized Text by Word Using words(): ')
print(lorenzo_paragraph.words())
```

Your output should display as shown below.

```
Tokenized Text by Word Using words():
Traceback (most recent call last):
print(lorenzo_paragraph.words())
AttributeError: 'str' object has no attribute 'words'
```

▼
What does the error mean?


```
Tokenized Text by Word Using words():
Traceback (most recent call last):
print(lorenzo_paragraph.words())
AttributeError: 'str' object has no attribute 'words'
```


The reader function called `words` is only designed to be used on corpora.

When we need to break non-corpus text data down by word, we tokenize using the `word_tokenize` method.

debugging

Important: Before moving on, make sure to comment out or delete the following lines from your code.

```
print(lorenzo_paragraph.words())
```

Otherwise, the computer will get stuck on this error when running the remaining exercises on this page.

Sentence Tokens

Using the `nltk.tokenize` package, we can import the `sent_tokenize` method from the NLTK library.

We will continue to work with the string of data assigned to the variable `lorenzo_paragraph`, as shown in the code below.

```
from nltk.tokenize import sent_tokenize

lorenzo_paragraph = "Lorenzo di Piero de'Medici was an Italian statesman, banker, de facto
        ruler of the Florentine Republic and the most powerful and enthusiastic patron of
        Renaissance culture in Italy. Also known as Lorenzo the Magnificent (Lorenzo il
        Magnifico) by contemporary Florentines, he was a magnate, diplomat, politician and
        patron of scholars, artists, and poets. As a patron, he's best known for his
        sponsorship of artists such as Botticelli and Michelangelo. He held the balance of
        power within the Italic League, an alliance of states that stabilized political
        conditions on the Italian peninsula for decades, and his life coincided with the
        mature phase of the Italian Renaissance and the Golden Age of Florence."
```

The imported `sent_tokenize` method allows us to break this text down by sentence, where each sentence is grouped in its own string within a list.

Enter the code below, then select the `TRY IT` button to run the file.

```
print(f'Tokenized Text by Sentence Using sent_tokenize():
        {sent_tokenize(lorenzo_paragraph)}')
```

Your output should display as shown below:

```
Tokenized Text by Sentence Using sent_tokenize(): ["Lorenzo di Piero de'Medici was an
Italian statesman, banker, de facto ruler of the Florentine Republic and the most powerful
and enthusiastic patron of Renaissance culture in Italy.", 'Also known as Lorenzo the
Magnificent (Lorenzo il Magnifico) by contemporary Florentines, he was a magnate, diplomat,
politician and patron of scholars, artists, and poets.', "As a patron, he's best known for
his sponsorship of artists such as Botticelli and Michelangelo.", 'He held the balance of
power within the Italic League, an alliance of states that stabilized political conditions
on the Italian peninsula for decades, and his life coincided with the mature phase of the
Italian Renaissance and the Golden Age of Florence.']
```

challenge

Try this variation:

Write the code to produce the following expected output, then select the `TRY IT` button to run the file and check your work.

The expected output is shown below.

```
Tokenized Text by Sentence, Separating Each Sentence by Line:
- Lorenzo di Piero de'Medici was an Italian statesman, banker, de facto ruler of the Florentine Republic and the most powerful and enthusiastic patron of Renaissance culture in Italy.
- Also known as Lorenzo the Magnificent (Lorenzo il Magnifico) by contemporary Florentines, he was a magnate, diplomat, politician and patron of scholars, artists, and poets.
- As a patron, he's best known for his sponsorship of artists such as Botticelli and Michelangelo.
- He held the balance of power within the Italic League, an alliance of states that stabilized political conditions on the Italian peninsula for decades, and his life coincided with the mature phase of the Italian Renaissance and the Golden Age of Florence.
```

▼
Solution

One possible solution is shown below, where we can use a `for` loop along with `sent_tokenize` to tokenize the text by sentence, separating each sentence by line.

```
print('\nTokenized Text by Sentence, Separating Each Sentence by Line:')
for element in sent_tokenize(lorenzo_paragraph):
  print(f'- {element}')
```

Access Text from the Web

Extract Text from the Web

NLP is not only limited to analyzing text found in a pre-existing corpus. We can scrap text from a website and then process it. In this example, we are going to use an article from the BBC. Before we can start extracting text from the web, we need to first install the BeautifulSoup package.

```
python3 -m pip install beautifulsoup4==4.11.1
```

We are ready to begin coding. Open the `web_text.py` file with the button below.

Import the processes we will use to extract text in the exercises on this page.
- Using the `urllib.request` package, we can import `urlopen`.
- Using the `bs4` package, we can import `BeautifulSoup`.
- Using the `nltk.tokenize` package, we can import `word_tokenize`.

```
from urllib.request import urlopen
from bs4 import BeautifulSoup
from nltk.tokenize import word_tokenize
```

Create some variables used for the various stages of parse text from a website.
- We first feed the computer the specified URL, which we called `extinction_url`.
- Second, we used the `urlopen` method to retrieve the raw text from the variable's URL page, then store its value in the variable called `extinction_html`.
- Finally, we created a third variable called `extinction_html_parse` to use the `BeautifulSoup` `'html_parser'` on the variable called `extinction_html`.

```
extinction_url = 'https://www.bbc.com/news/science-environment-
        61242789'
extinction_html = urlopen(extinction_url)
extinction_html_parse = BeautifulSoup(extinction_html,
        'html.parser')
```

Extracting text is done by HTML tag. We are going to use the <p> tag along with the `find_all` and `get_text` methods. This will return the text from all of the paragraphs on the website.

Enter the code below, then select the `TRY IT` button to run the file.

```
for index, element in
        enumerate(extinction_html_parse.find_all('p')):
  words = element.get_text()
  print(f'\nTokens in Paragraph {index + 1}:
        {word_tokenize(words)}')
```

Articles on the internet can change over time, so your output may differ from the one shown below. Your output should be, for the most part, similar. For brevity's sake, not all of the paragraphs are show.

```
Tokens in Paragraph 1: ['By', 'Helen', 'BriggsEnvironment',
'correspondent']
Tokens in Paragraph 2: ['One', 'in', 'five', 'reptiles', 'is',
'threatened', 'with', 'extinction', ',', 'according', 'to',
'the', 'first', 'comprehensive', 'assessment', 'of', 'more',
'than', '10,000', 'species', 'across', 'the', 'world', '.']
Tokens in Paragraph 3: ['Scientists', 'are', 'calling', 'for',
'urgent', 'conservation', 'action', 'for', 'crocodiles', 'and',
'turtles', ',', 'which', 'are', 'in', 'a', 'particularly',
'dire', 'situation', '.']
Tokens in Paragraph 4: ['They', 'say', 'reptiles', 'have',
'long', 'been', 'overlooked', 'in', 'conservation', ',',
'because', 'they', 'are', 'seen', 'as', 'less', 'charismatic',
'than', '``', 'furry', 'and', 'feathery', "''", 'creatures',
'.']
Tokens in Paragraph 5: ['So', 'far', ',', '31', 'species',
'have', 'gone', 'extinct', '.']
Tokens in Paragraph 6: ['The', 'study', ',', 'published', 'in',
'Nature', ',', 'took', 'more', 'than', '15', 'years', 'to',
'complete', ',', 'because', 'of', 'problems', 'getting',
'funding', 'for', 'the', 'work', '.']
```

challenge

Try this variation:

Write the code to produce the following expected output, then select the `TRY IT` button to run the file and check your work.

The expected output is shown below.

```
Tokens in Bolded Text: 1: ['One', 'in', 'five', 'reptiles',
'is', 'threatened', 'with', 'extinction', ',', 'according',
'to', 'the', 'first', 'comprehensive', 'assessment', 'of',
'more', 'than', '10,000', 'species', 'across', 'the',
'world', '.']
```

▼ **Solution**

One possible solution is shown below, where we can use a `for` loop along with `find_all` and `get_text` to return all words in the `<b>` tags.

```
for index, element in
        enumerate(extinction_html_parse.find_all('b')):
  words = element.get_text()
  print(f'\nTokens in Bolded Text: {index + 1}:
        {word_tokenize(words)}')
```

Extract Text from Local Files

We can also apply the same NLP concepts to text files on our local computer. The `local_article.txt` file resides in the `exercises` directory. Be sure to import the `word_tokenize` function so we can tokenize the text.

Enter the code below, then select the `TRY IT` button to run the file.

```
from nltk.tokenize import word_tokenize

with open('exercises/local_article.txt') as local_text_file:
  raw_local_text_file = local_text_file.read()
  print(f"\nTokens from the Local Article Called
        'local_article.txt':
        {word_tokenize(raw_local_text_file)}")
```

Your output should display as shown below:

```
Tokens from the Local Article Called 'local_article.txt':
['One', 'in', 'five', 'reptiles', 'is', 'threatened', 'with',
'extinction', ',', 'according', 'to', 'the', 'first',
'comprehensive', 'assessment', 'of', 'more', 'than', '10,000',
'species', 'across', 'the', 'world', '.', 'Scientists', 'are',
'calling', 'for', 'urgent', 'conservation', 'action', 'for',
'crocodiles', 'and', 'turtles', ',', 'which', 'are', 'in', 'a',
'particularly', 'dire', 'situation', '.', 'They', 'say',
'reptiles', 'have', 'long', 'been', 'overlooked', 'in',
'conservation', ',', 'because', 'they', 'are', 'seen', 'as',
'less', 'charismatic', 'than', '``', 'furry', 'and', 'feathery',
"''", 'creatures', '.', 'So', far', ',', '31', 'species',
'have', 'gone', 'extinct', '.', 'The', 'study', ',',
'published', 'in', 'Nature', ',', 'took', 'more', 'than', '15',
'years', 'to', 'complete', ',', 'because', 'of', 'problems',
'getting', 'funding', 'for', 'the', 'work', '.']
```

challenge

Try this variation:

We can also extract tokens from user input.

Enter the below code, then select the `TRY IT` button to run the file. Codio will open the terminal where you can type several words (separated by spaces). Press `ENTER` when done.

```
from nltk.tokenize import word_tokenize

user_input = input('Enter Some Random Text: ')
print(f'You typed {len(word_tokenize(user_input))} words.')
```

Your output will look something like this:

```
Enter Some Random Text: cat dog meow
You typed 3 words.
```